

reverse 정리

범위 안의 원소 순서를 반대로 뒤집는 알고리즘입니다.

사용처 (Where)

- 배열/문자열 뒤집기
- 역순 처리가 필요할 때
- 팰린드롬 검사 전후 비교

방법 (How to Use)

- begin/end 범위를 넘김
- 원본 순서가 직접 바뀜
- string, vector, deque 등에 사용
- 부분 범위도 뒤집을 수 있음

기본 정보

- 필요 헤더: `#include <algorithm>`
- 기본 선언: `reverse(v.begin(), v.end());`

왜 써야 할까? (Why)

- 직접 swap 반복문을 작성하지 않아도 됨
- 부분 범위 뒤집기도 쉽게 가능
- 문자열 문제에서 자주 사용

주요 함수

함수/문법	의미
<code>reverse(first,last)</code>	범위 뒤집기
<code>rbegin/rend</code>	역순 순회
<code>reverse_copy</code>	뒤집은 결과 복사

기본 코드 예시

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {1, 2, 3};
    reverse(v.begin(), v.end());

    for (int x : v) cout << x << ' ';
}
```

reverse 비교와 응용

STL을 직접 구현이나 C 스타일 코드와 비교하고, 실제 문제에서 쓰는 방식을 확인합니다.

시간 복잡도

동작	복잡도
reverse	$O(n)$
역순 순회	$O(n)$
추가 메모리	거의 없음

STL vs 직접 구현 vs C 스타일

구분	STL	직접 구현	C 스타일
개발 난이도	reverse 함수 호출만 알면 됨	반복문과 조건을 직접 설계	qsort/직접 반복문 사용
코드 길이	짧고 의도가 분명함	길어지고 실수 지점 증가	자료형 처리와 캐스팅이 번거로움
안정성	표준 라이브러리라 검증됨	구현 실수 가능	포인터/범위 실수 가능
추천 상황	대부분의 일반 상황	원리를 공부하거나 특수 최적화	C 코드와 연동할 때

응용: 문자열 팰린드롬 검사

원본과 뒤집은 문자열을 비교하여 앞뒤가 같은지 확인합니다.

```
#include <iostream>
#include <string>
#include <algorithm>
using namespace std;

int main() {
    string s = "level";
    string t = s;
    reverse(t.begin(), t.end());

    cout << (s == t);
}
```

처음 배울 때 자주 하는 실수

- reverse는 원본을 바꿈
- 원본 보존이 필요하다면 복사본에 사용
- end는 마지막 다음 위치라는 점을 기억

비슷한 항목과 차이

- rbegin/rend는 원본을 바꾸지 않고 역순 순회
- sort는 크기 순서로 정렬
- rotate는 일부를 회전

한 줄 정리: reverse은(는) '배열/문자열 뒤집기' 상황에서 먼저 떠올리면 좋은 STL입니다.